Query Engines

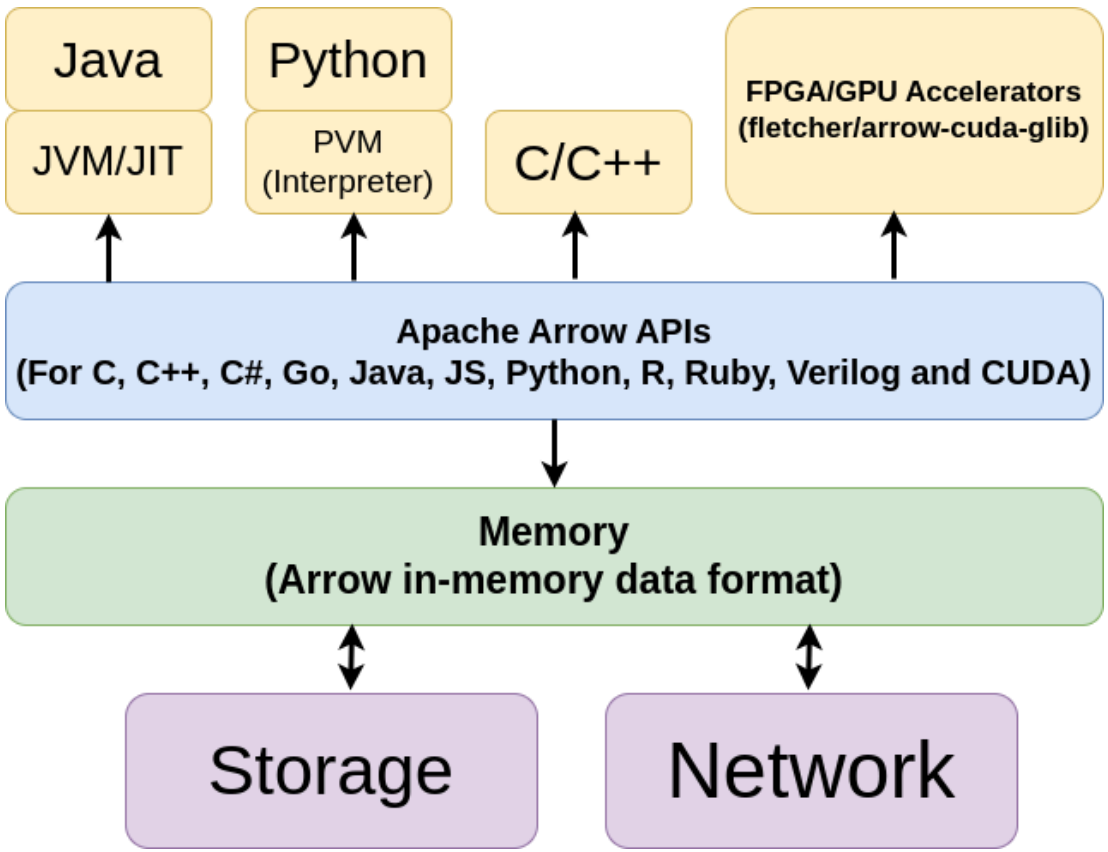
Hermilo

17 de Febrero de 2026

Apache Arrow

- El proyecto **Apache Arrow** es un gran esfuerzo dirigido a resolver los problemas fundamentales de la analítica de datos.
- Apache Arrow define un **formato de memoria columnar** estandarizado e independiente del lenguaje, diseñado explícitamente para facilitar el análisis de datos de alto rendimiento y el intercambio de información.
- Su innovación clave reside en una estructura en memoria cuidadosamente diseñada y optimizada para las arquitecturas de CPU modernas, lo que permite el acceso a datos sin copias (**zero-copy**), el uso eficiente de instrucciones SIMD y una interoperabilidad fluida entre diversos motores de procesamiento de datos y lenguajes de programación.

¿Qué es Apache Arrow?



APACHE ARROW

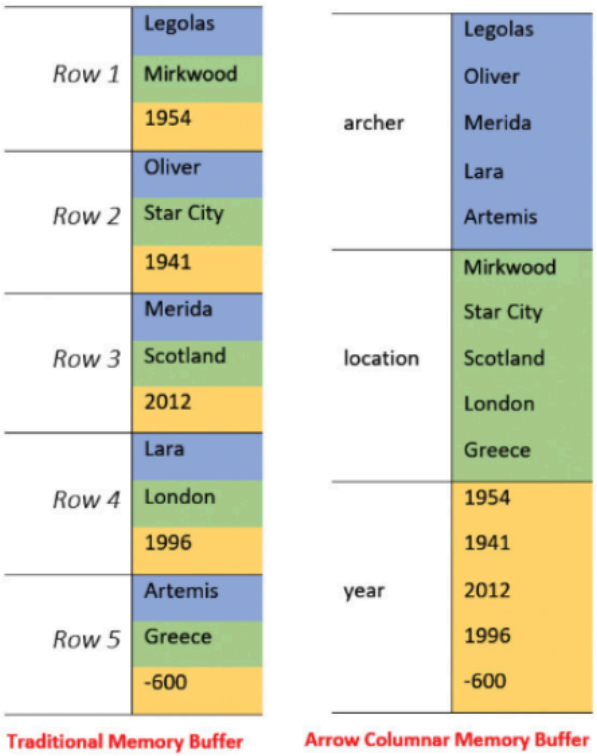
- Estandar de intercambio de datos.
- Un formato in-memory.
- Un formato de Red.
- Un formato de Almacenamiento.
- Biblioteca de I/O
- Biblioteca de cómputo vectorizado.
- Biblioteca de manejo de dataframes.
- Un query engine.
- Administrador de datasets particionados.

Apache Arrow

- Apache Arrow nació como un **Formato Columnar de Datos**.
- El núcleo del modelo de memoria de Arrow es la **estructura de datos columnar**, que organiza la información en búferes de memoria contiguos, cada uno de los cuales representa una columna lógica.
- Esta disposición contrasta con el almacenamiento tradicional basado en filas al permitir operaciones vectorizadas y mejorar la eficiencia de la caché.

¿Qué es Columnar?

	ARCHER	LOCATION	YEAR
ROW 1	Legolas	Mirkwood	1954
ROW 2	Oliver	Star City	1941
ROW 3	Merida	Scotland	2012
ROW 4	Lara	London	1996
ROW 5	Artemis	Greece	-600



¿Qué es Columnar?

Con el formato columnar se aprovecha :

- Localidad de Memoria.
- I/O
- Vectorización

- **Consulta** : Todos los arqueros en Europa, **solo se necesitan dos columnas!**. Menos operaciones de I/O, menos uso de memoria.
- **Consulta** : Calcula la media para la columna Year, **solo se necesita una columna!**
  - Vectorizar operaciones requiere memoria contigua.
  - La columna ya se encuentra en memoria contigua!

PyArrow

- **PyArrow** es el punto de entrada al ecosistema de Apache Arrow para desarrolladas de Python, y es el medio que ofrece el acceso a los muchos beneficios de Arrow.
- De manera que el tipo fundamental de datos en PyArrow es una **columna de datos** la cual es expuesta por mendio de un objeto `pyarrow.Array`.
- A este nivel, PyArrow es similar a los arreglos 1D de **NumPy**.

PyArrow Arrays

```python
import pyarrow as pa

## Arrays can be made of numbers
>>> pa.array([1, 2, 3, 4, 5])

## Or strings
>>> pa.array(["A", "B", "C", "D", "E"])

## Or even complex objects
>>> pa.array([{"a" : 5}, {"b" : 10}])

## Arrays can also be masked
>>> pa.array([1, 2, 3, 4, 5],
mask = pa.array([True, False, True, False,
True]) )
```

Comparado a los arreglos clásicos de NumPy, los arreglos de PyArrow son un poco más complejos.

- Estandar de intercambio de datos.

Manipulación de Arreglos

- La comunidad Arrow ha creado una implementación de referencia de código abierto de un motor de cálculo y consulta basado en el formato Arrow llamado **Acero**.
- PyArrow proporciona también acceso al motor de cálculo **Acero** mediante el módulo `pyarrow.compute`.
- Para este fin, existe la biblioteca Acero para facilitar diversas implementaciones de alto rendimiento de funciones que operan con datos con formato Arrow, junto con la creación de planes de ejecución de filtrados, agregaciones y transformaciones para flujos de datos.

```python
import pyarrow.compute as pc

>>> arr = pa.array([1, 2, 3, 4, 5])
>>> pc.multiply(arr, 2)
>>> pc.value_counts(arr)
>>> pc.min_max(arr)
```

PyArrow Tables

- Como los arreglos son “columnas”, estos puden ser agrupados para formar `pyarrow.Table`.
- Las tablas son constituidas por `pyarrow.ChunkedArray` de manera que concatenar filas a ellas resultan una operación barata.
- A este nivel, PyArrow es similar a los **DataFrames de Pandas**.

PyArrow Tables

```python
>>> table = pa.table([
    pa.array([1, 2, 3, 4, 5]),
    pa.array(["a", "b", "c", "d", "e"]),
    pa.array([1.0, 2.0, 3.0, 4.0, 5.0]),
], names = ["col1", "col2", "col3"])
>>> table.take([0, 1, 4])
col1: [[1,2,5]]
col2: [["a","b","e"]]
col3: [[1,2,5]]
>>> table.schema
col1: int64
col2: string
col3: double
```

- Comparado a Pandas, las tablas de PyArrow son completamente implementadas en C++ y nunca modifican datos **in-place**.
- `Tables` están basadas en **ChunkedArrays** de manera que concatenar datos es una operación **zero copy**.
- El motor de cómputo Acero en Arrow es capaz de proporcionar muchas de las operaciones comunes de la analítica de datos como joining, filtrados y agregaciones de datos.

Ejecutando Analítica de Datos

- El motor de cómputo de Acero potencia las capacidades de análisis y transformación disponibles en las tablas
- Muchas funciones de `pyarrow.compute` proporcionan kernels que operan en tablas y las `Table` exponen métodos de join, filtrado y agrupación.

```python
import pyarrow as pa
import pyarrow.compute as pc

>>> table = pa.table([
  pa.array(["a", "a", "b", "b", "c", "d", "e", "c"]),
  pa.array([11, 20, 3, 4, 5, 1, 4, 10])
], names = ["keys", "values"])
>>> table.filter(pc.field("values") == )
>>> table.group_by("keys").aggregate(["keys", "sum"])
>>> table1 = pa.table({"id" : [1, 2, 3],
  "year" : [2020, 2022, 2019]
})
>>> table2 = pa.table({"id" : [3, 4],
  "n_legs" : [5, 100],
  "animal" : ["Brittle Stars", "Centipede"]
})
>>> table1.join(table2, keys = "id")
```

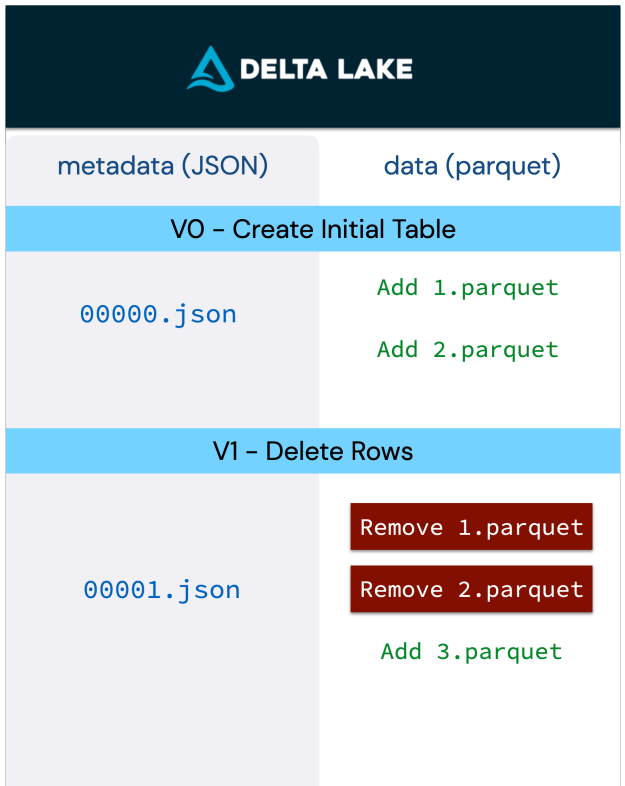
Apache Parquet, a columnar storage format

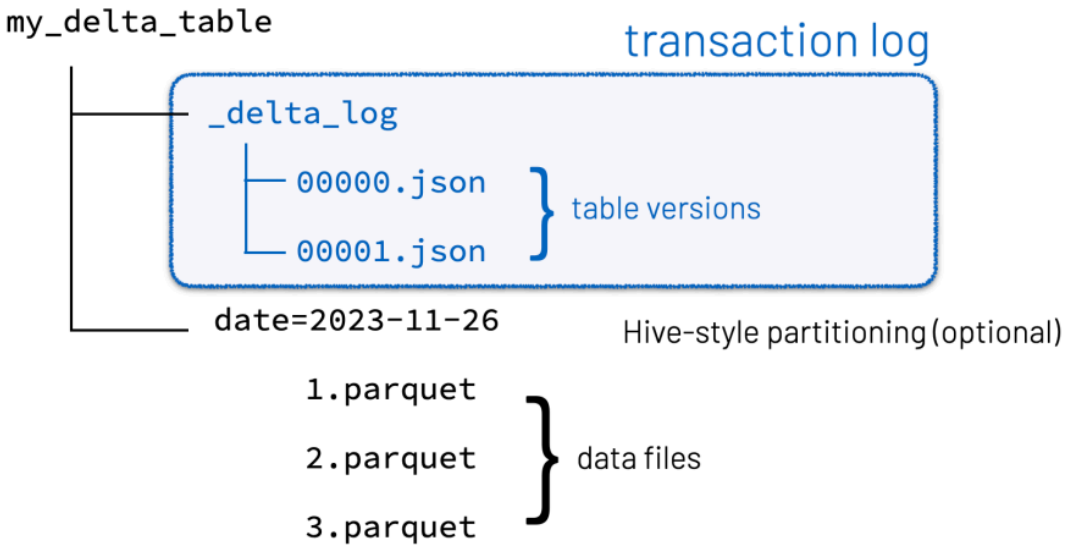


Parquet, a columnar storage format

- Parquet almacena los datos en columnas en lugar de en filas, a diferencia de los formatos de almacenamiento tradicionales basados en filas (como CSV o JSON).
- Este diseño columnar implica que, cuando una consulta solo necesita acceder a unos pocos campos o columnas específicos de un conjunto de datos, Parquet puede leer únicamente las columnas relevantes sin cargar la fila completa en la memoria.
- Parquet también emplea diversas técnicas para mejorar aún más el rendimiento y reducir los costes de almacenamiento:
  - Compression
  - Encoding
  - Metadata and statistics

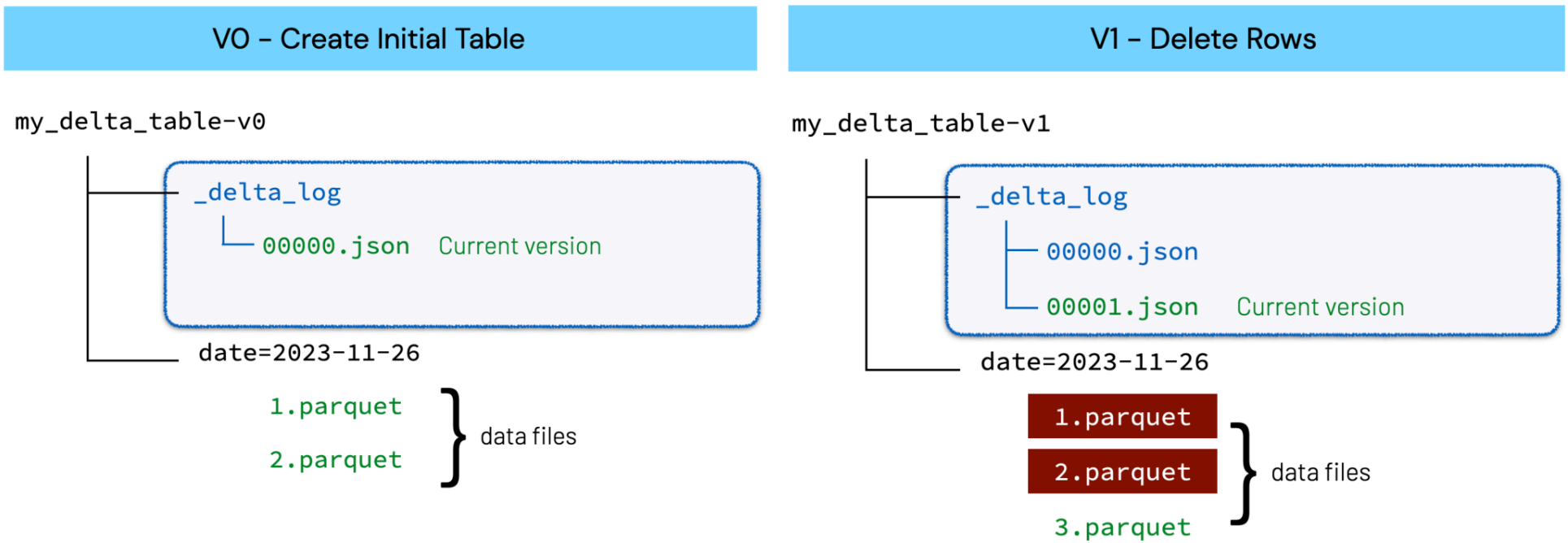
Data Lakes hacen uso intensivo de archivos Parquet.





Tomado de Denny Lee, “Understanding the Delta Lake Transaction Log at the File Level”, Denny Lee (blog), November 26, 2023.

Data Lakes hacen uso intensivo de archivos Parquet.



Tomado de Denny Lee, “Understanding the Delta Lake Transaction Log at the File Level”, Denny Lee (blog), November 26, 2023.

Query Engines

- Los **Query Engines**[1] incorporan patrones arquitectónicos sofisticados, diseñados para transformar consultas declarativas en planes de ejecución eficientes que aprovechan el hardware subyacente y las estructuras de datos.

[1] Motores de procesamiento de consultas

Query Engines

- Un **Query Engine** transforma una consulta (como una consulta SQL) en resultados concretos a través de varias etapas:
  - **Parsing (Análisis sintáctico)** : Convierte el texto de la consulta en una representación estructurada (como un árbol de sintaxis abstracta)
  - **Planificación**: Determinar qué operaciones son necesarias (escaneo, filtrado, unión, agregación)
  - **Optimización**: Reordenar y transformar las operaciones para lograr eficiencia
  - **Ejecución** : Procesar los datos y generar los resultados.

Es posible que este pipeline recuerde a un compilador, y eso no es casualidad. Los motores de consulta son, en esencia, **compiladores especializados que traducen consultas declarativas en planes de ejecución eficientes**.

Ejemplo

Veamos una consulta ligeramente más compleja:

```sql
SELECT department,
AVG(salary)
FROM employees
WHERE hire_date > '2020-01-01'
GROUP BY department
ORDER BY AVG(salary) DESC;
```

Esta Consulta:

- Escanea la tabla `employee`.
- Filtra sólo los recién contratados.
- Agrupa a los empleados por departamento.
- Calcula el salario promedio por departamento.
- Ordena los resultados por ese promedio.

Un query engine **debe determinar la forma más eficiente de ejecutar estas operaciones**. ¿Debe filtrar antes o después de agrupar? ¿Cómo debe almacenar los resultados intermedios? **Estas decisiones repercuten significativamente en el rendimiento, especialmente con grandes conjuntos de datos**.

SQL: The Universal Query Language

SQL (Structured Query Language) ha sido el lenguaje de consultas dominante desde 1970. Lo podemos encontrar en:

- Bases de Datos Relacionales (PostgreSQL, MySQL, SQLite)
- Data warehouses (Snowflake, BigQuery, Redshift)
- Sistemas de Big data (Apache Spark, Presto, Hive)
- Analítica embebida (DuckDB)

Aquí hay dos ejemplos más que muestran la expresividad de SQL:

Identificación de las 5 páginas más visitadas ayer:

```sql
SELECT page_url, COUNT(*) AS visits
FROM page_views
WHERE view_date = CURRENT_DATE - 1
GROUP BY page_url
ORDER BY visits DESC
```

Cálculo del crecimiento mes a mes:

```sql
SELECT month, revenue,
       revenue - LAG(revenue) OVER (ORDER BY month) AS growth
FROM monthly_sales
WHERE year = 2024;
```

```
6  LIMIT 5;
```

Más allá de SQL : DataFrames APIs

- Si bien SQL es omnipresente, muchos query engines también ofrecen API programáticas.
- Estas son especialmente populares en la ciencia de datos, donde las consultas a menudo se construyen de forma dinámica o se combinan con código personalizado.
- Esta es la misma consulta expresada utilizando la API de DataFrame de Polars en Python:

```python
import polars as pl

pl.read_parquet(
        "/data/employees"
    ).filter(
        pl.col("hire_date") > "2020-01-01"
    ).group_by(
        pl.col("department")
    ).agg(
        pl.col("salary").avg().alias("avg_salary")
    ).sort_by(
        "avg_salary",
        descending = True
    )
```

La API de Polars ofrece las mismas operaciones lógicas que SQL, pero expresadas como llamadas a métodos. Internamente, ambos enfoques generan el mismo plan de consulta.

Polars : Query Engine for DataFrames

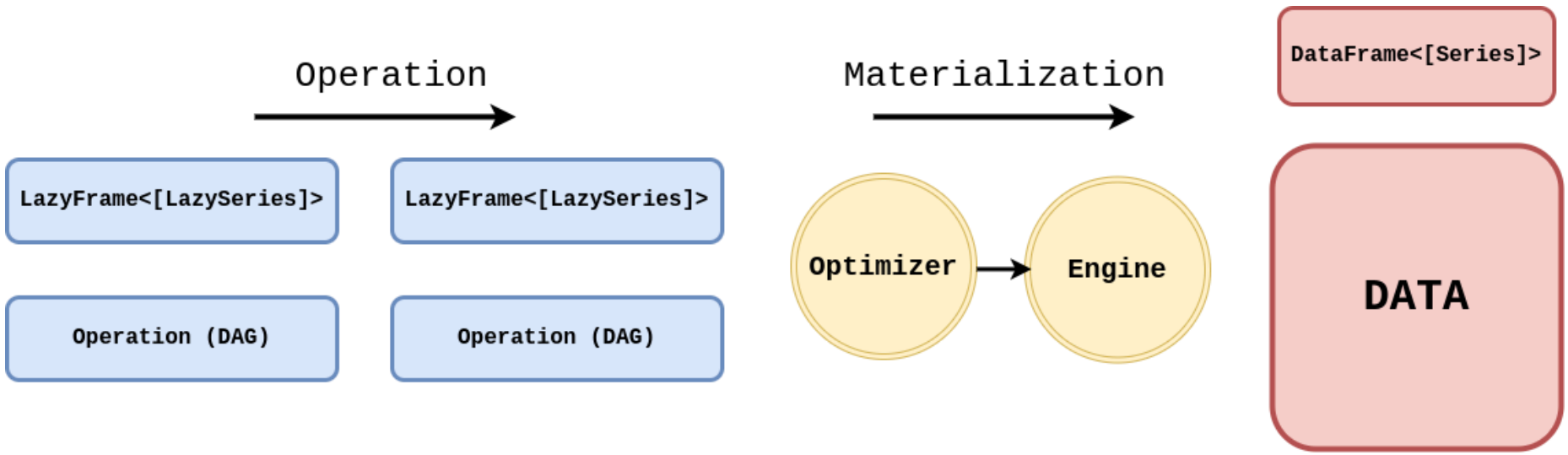


Polars : Query Engine for DataFrames

Polars es una biblioteca desarrollada en Rust que integra las siguientes características por diseño:

- **Velocidad** : Rust es un lenguaje de system programmig conocido por su desempeño y seguridad.
- **Paralelismo** : Aprovecha arquitecturas multicore he implementa algoritmos paralelos de work stealing.
- **Eficiencia de memoria**:
  - Polars utiliza evaluaciones lazy, lo que significa que una operación no es realizada hasta que esta es necesitada.
  - Las consultas pueden ser encadenadas y optimizadas antes de su ejecución, lo que se traduce en ejecuciones más eficientes de queries.
- **Almacenamiento eficiente de datos** : Polars utiliza Apache Arrow como modelo de almacenamiento en memoria. Es decir, utiliza un formato columnar del almacenamiento de datos, lo cual resulta más eficiente que el tradicional almacenamiento basado en filas (como el utilizado por Pandas).
- Diseñado para **out-of-core processing** (más grande que la RAM).

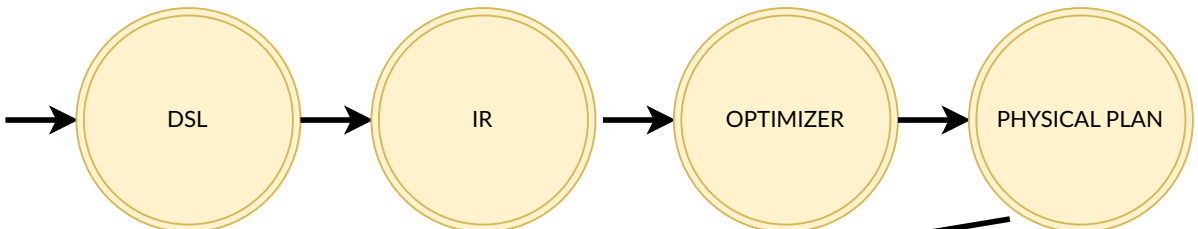
Polars : Query Engine for DataFrames



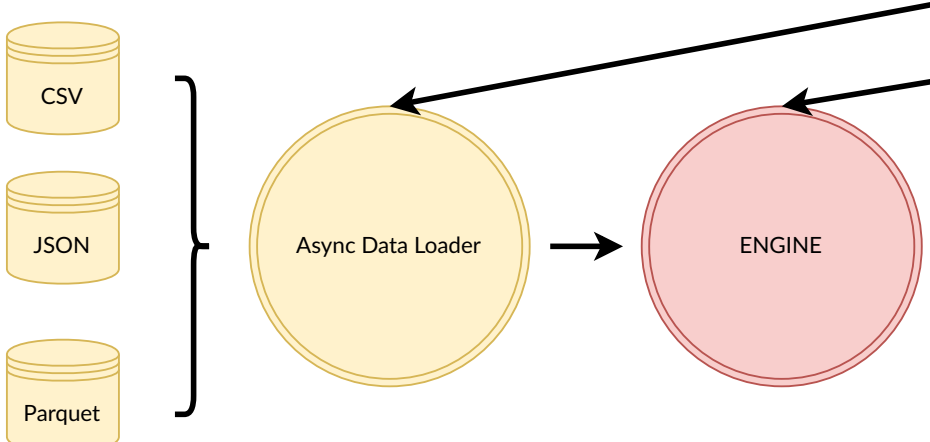
Roadmap de un query

Logical

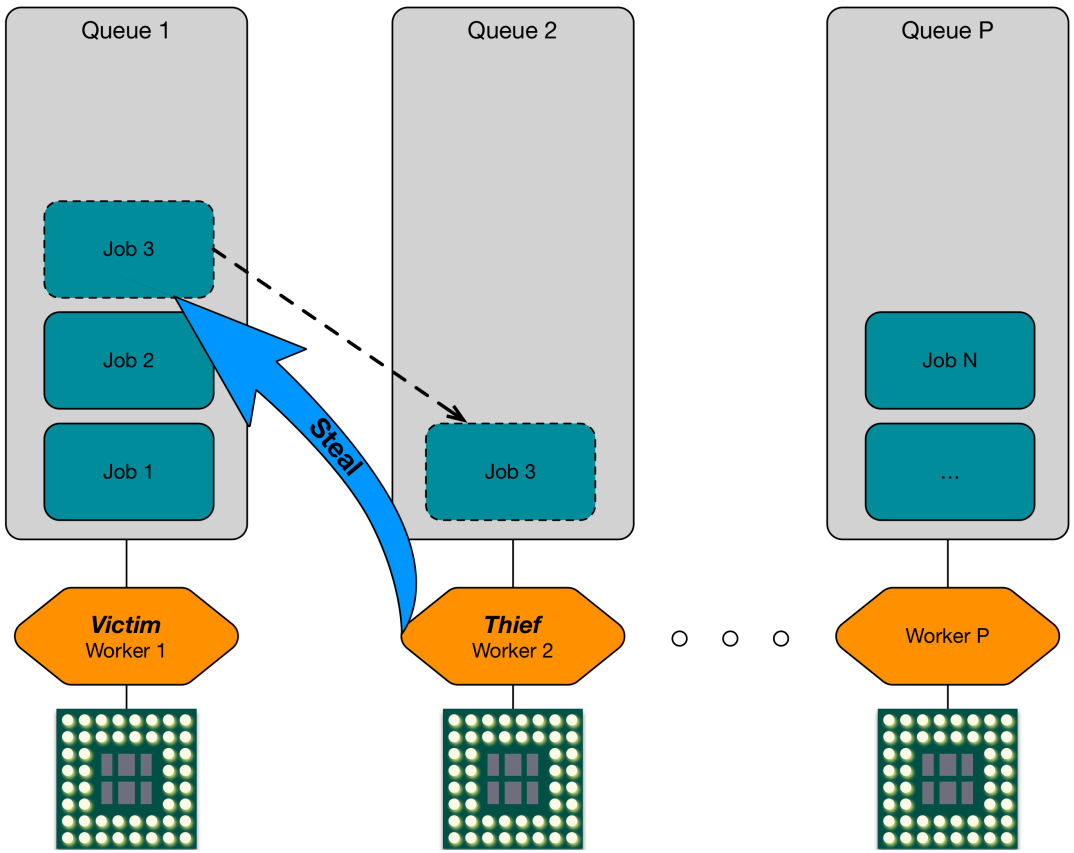
```python
(
  pl.scan_parquet("my_file.parquet")
  .filter(pl.col("dogs")=="Retriever")
  .select("dogs", "origins","weight")
  .head(10)
  .collect(engine="gpu")
)
```



Physical



Paralelismo : Work Stealing



Polars Expressions

¿Qué es una expresión?

Una expresión es un árbol de operaciones que describen cómo construir ona o más Series.

DuckDB : a SQL database that runs everywhere



DuckDB : a SQL database that runs everywhere

- DuckDB es un sistema de bases de datos analíticas de alto rendimiento.
- Está diseñado para ser rápido, fiable, portable y fácil de usar.
- DuckDB ofrece un dialecto SQL completo, con funcionalidades que van mucho más allá del SQL básico.
- DuckDB está disponible como una aplicación CLI standalone y clientes para Python, R, Java, etc.

SQL

```sql
-- Get the top-3 busiest train stations
SELECT
    station_name,
    count(*) AS num_services
FROM train_services
GROUP BY ALL
ORDER BY num_services DESC
LIMIT 3;
```

Python

```python
# Get the top-3 busiest train stations
import duckdb
duckdb.sql("""
    SELECT station, count(*) AS num_services
    FROM train_services
    GROUP BY ALL
    ORDER BY num_services DESC
    LIMIT 3;
    """)
```

DuckDB : Importación de datos

Lectura de archivos CSVs

```sql
-- Load CSV file to a table. DuckDB auto-detects
-- the CSV's format, column name and types
CREATE TABLE stations AS
    FROM 'https://blobs.duckdb.org/stations.csv';
```

Lectura directa de un archivo Parquet sobre HTTPS

```sql
-- Directly query Parquet file over HTTPS
SELECT
    station_name,
    count(*) AS num_services
FROM 'https://blobs.duckdb.org/train_services.parquet'
GROUP BY ALL
ORDER BY num_services DESC
LIMIT 10;
```

Apache DataFusion : An extensible query engine written in Rust that uses Apache Arrow as its in-memory format.

